Chapter 5

Introduction to Design Patterns

Chapter 5: Introduction to Design Patterns (Part I)

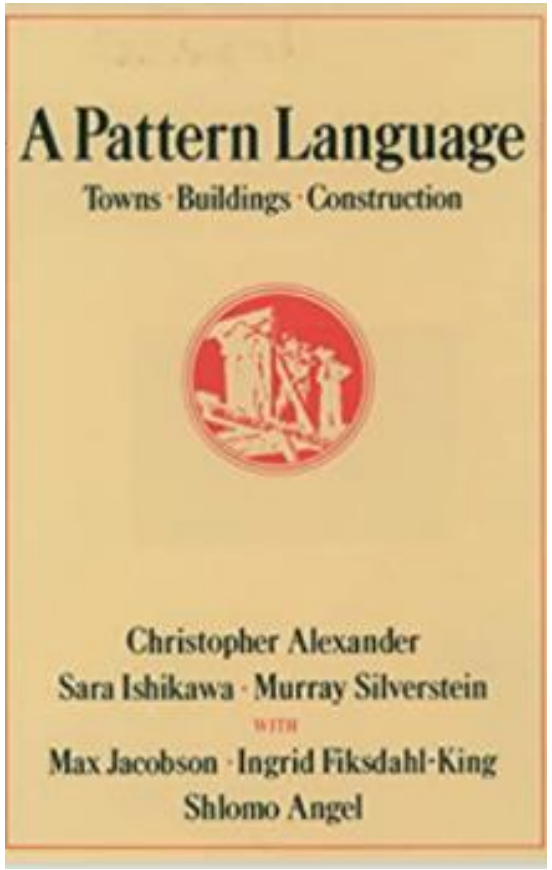
- History
- Object-oriented design principles
- Design patterns

The history of design patterns

The term "pattern" originated in the field of architecture

In 1977, Christopher Alexander, a renowned United States architect and director of the Center for Environmental Structures at the University of California, Berkeley, described some common architectural design problems in his book A *Pattern Language: Towns Building Construction*, and proposed 253 questions about the impact on towns, neighborhoods, the basic pattern of design for homes, gardens, rooms, etc.

In 1979, another of his classic books, The Timeless Way of Building, further strengthened the idea of design patterns and pointed the way for later architectural design.



The history of design patterns

"Each pattern describes a problem that is constantly recurring around us, and the core of the solution to that problem. This way, you'll be able to use the solution again and again without having to reinvent the wheel. ”

-- Christopher Alexander

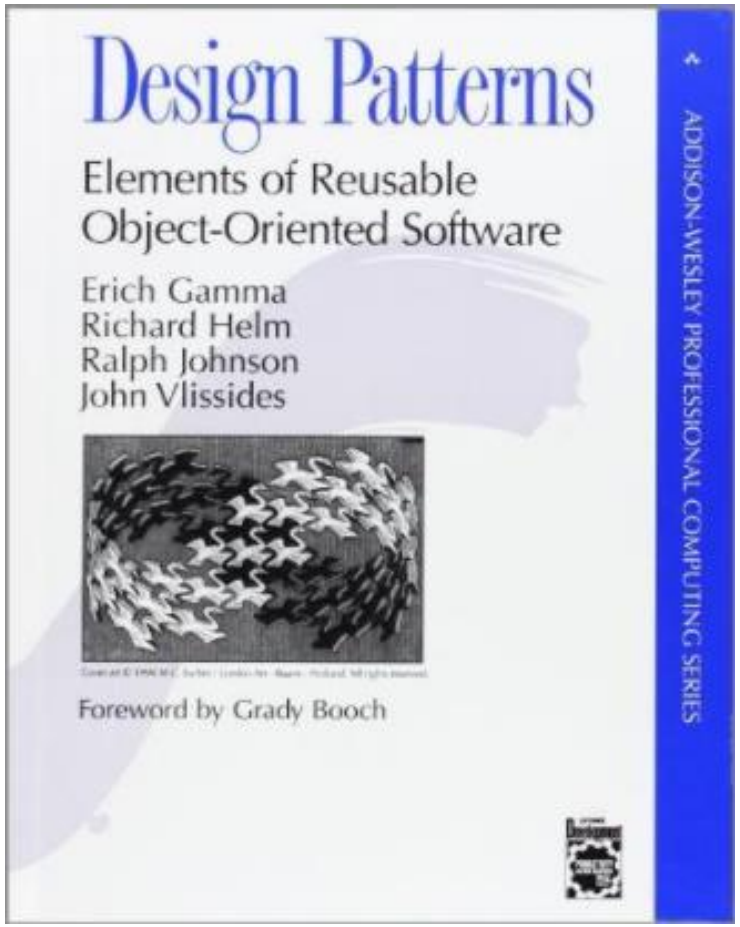
The core of the pattern:

Provide solutions to similar problems.

The history of design patterns

"Patterns" have their origins in architecture, but the same applies to object-oriented software design. In software development, we replace walls, doors and windows with objects and interfaces.

- In 1995, Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides co-authored a book called Design Patterns - Elements of Reusable Object-Oriented Software.
- The book contains 23 design patterns and is the first to mention the concept of design patterns in software development. This was a milestone event in the field of design patterns, leading to a breakthrough in software design patterns.
- The four authors are collectively known as GOF (Gang of Four).

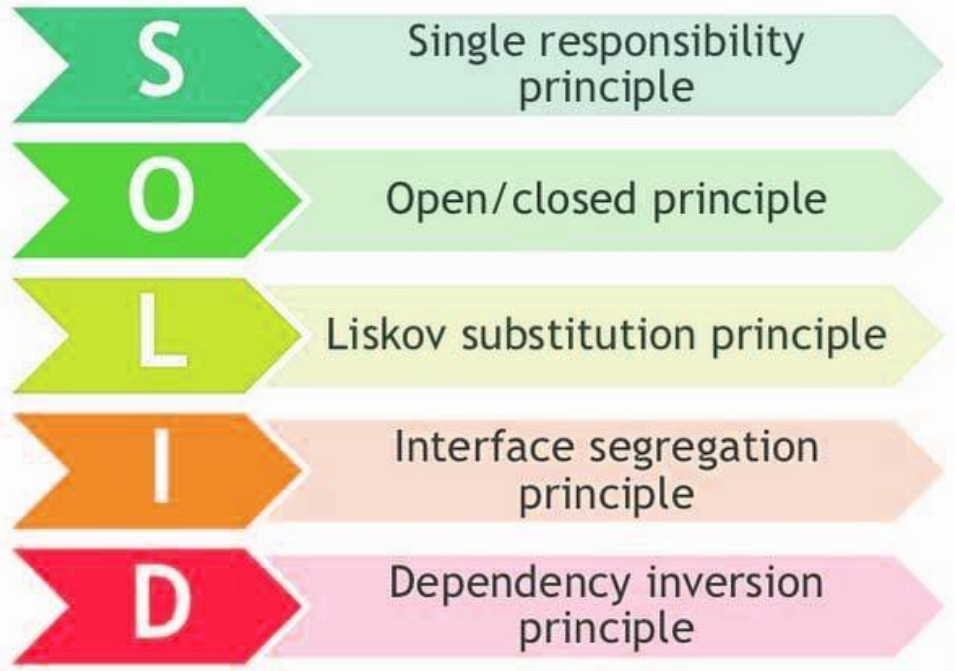


Chapter 5: Introduction to Design Patterns (Part I)

- History
- Object-oriented design principles
- Design patterns

Object-oriented design principles

- There are common principles to follow when designing object-oriented programs, including:



Single responsibility principle

- **The Single Responsibility Principle (SRP) defines:**
- **For a class, there should be only one reason for its change.**
  - Don't group different responsibilities, as changes can affect unrelated responsibilities.
  - If a class has too many responsibilities, the less likely it is to be reused, the more likely it is to couple those responsibilities together, and a change in one responsibility may weaken and inhibit the class's ability to accomplish other duties.
  - If there is more than one motivation to change a class, then the class has more than one responsibility, and the separation of duties of the class should be considered.

Open-closed principle

- **Open-Closed Principle – OCP**
- **A software entity should be open to extensions and closed to modifications. That is, when designing a module, the module should be extended without modification.**
  - By extending existing software systems, new behaviors can be provided to meet new requirements for software, so that changing software systems have a certain degree of adaptability and flexibility.
  - In object-oriented programming, through abstract classes and interfaces, the features of concrete classes are specified as abstraction layers, which are relatively stable and do not need to be changed, so as to meet the requirements of "closed to modifications ". Concrete classes derived from abstract classes can change the behavior of the system, thus satisfying the "open to extensions".

Liskvo Substitution Principle

- **Definition of the Liskvo Substitution Principle (LSP):**
- Objects of a subclass should be replaceable with objects of the superclass without breaking the program's behavior.
- The reverse substitution does not work, and if a software entity uses a subclass, then it does not necessarily apply to the base class.

Interface Segregation Principle

- **Interface Segregation Principle – ISP: The client should not depend on interfaces it does not need.**
  - It is better to use multiple specialized interfaces than to use a single master interface. That is, the dependencies of one class on another class should be built on the smallest interface.
  - Once an interface is too large, it needs to be split into smaller interfaces, and the client using the interface only needs to know the methods associated with it.

Dependency Inversion Principle

- **Definition of Dependency Inversion Principle (DIP):**
- **High-level modules shouldn't depend on low-level modules, they should all rely on abstractions. Program should focus on the interface, not the implementation.**
  - Instead of concrete classes, you should use interfaces and abstract classes for variable type declarations, parameter type declarations, method return type descriptions, and data type transformations.
  - The traditional design approach to process systems tends to make high-level modules dependent on low-level modules and abstract levels dependent on concrete levels. The inversion principle is to turn this false dependency upside down.

Composite/Aggregate Reuse Principle

- **Definition of the Composite/Aggregate Reuse Principle (CARP): Maximize the use of object composition, rather than inheritance for the purpose of reuse.**
  - Use some existing objects inside a new object to make them part of the new object; New objects are delegated to reuse these objects.
  - Composite/aggregation should be used first, and inheritance should be considered second. When using inheritance, the principle of LSP should be strictly followed. Effective use of inheritance will help to understand the problem and reduce complexity, while misuse of inheritance will increase the difficulty and complexity of the system in software construction and maintaining.

Law of Demeter

Law of Demeter – LoD:

A software entity should interact with other entities as little as possible.

- Each software unit has minimal knowledge of the others and is limited to those software units that are closely related to the unit.
- In this way, when one module is modified, it will affect the other modules as little as possible. Scaling will be relatively easy. This is a restriction on communication between software entities. It requires limiting the width and depth of communication between software entities.

Quiz

Which principles are violated, and how can you refactor the design?

```java
class EmployeeSystem {
    void calculateSalary() { /* salary logic */ }
    void generateReport() { /* report logic */ }
    void sendEmailToHR()  { /* email logic */ }
    double getBonus(String employeeType, double base) {
        if (employeeType.equals("Manager")) return base * 0.2;
        if (employeeType.equals("Engineer")) return base * 0.1;
        return 0;
    }
    class Employee {
        void work() { System.out.println("Working..."); }
    }
    class LazyEmployee extends Employee {
        @Override void work() {
            throw new UnsupportedOperationException("I don't work!");
        }
    }
    interface Device {
        void print();
        void scan();
        void fax();
    }
    class SimplePrinter implements Device {
        public void print() { System.out.println("Printing..."); }
        public void scan() { throw new UnsupportedOperationException(); }
        public void fax()  { throw new UnsupportedOperationException(); }
    }
    class EmailSender {
        void send(String msg) { System.out.println("Sending: " + msg); }
    }
    class HRNotifier {
        private EmailSender sender = new EmailSender();
        void notifyHR() { sender.send("Report sent!"); }
    }
}
```

Quiz

```
void calculateSalary() { /* salary logic */ }
void generateReport() { /* report logic */ }
void sendEmailToHR()  { /* email logic */ }
```

1. SRP (Single Responsibility)
2. Problem: EmployeeSystem mixes salary calc, reporting, and emailing.
3. Fix: Split by reasons to change.

•SalaryService.calculate()
•ReportService.generate()
•NotificationService.sendToHR()

Quiz

```
double getBonus(String employeeType, double base) {
    if (employeeType.equals("Manager")) return base * 0.2;
    if (employeeType.equals("Engineer")) return base * 0.1;
    return 0;
}
```

1. OCP (Open/Closed)
2. Problem: getBonus(String employeeType, ...) uses conditionals; adding a type changes existing code.
3. Fix: Use polymorphism.

```
interface BonusPolicy { double bonus(double base); }
class ManagerBonus implements BonusPolicy { public double bonus(double b){ return b*0.2; } }
class EngineerBonus implements BonusPolicy { public double bonus(double b){ return b*0.1; } }
```

Quiz

```
class Employee {
    void work() { System.out.println("Working..."); }
}
class LazyEmployee extends Employee {
    @Override void work() {
        throw new UnsupportedOperationException("I don't work!");
    }
}
```

1. LSP (Liskov Substitution)
2. Problem: LazyEmployee extends Employee but work() throws UnsupportedOperationException, breaking substitutability.
3. Fix options:

• Correct the hierarchy: don't extend if contract can't be honored (e.g., model NonWorkingAccount separately).

Quiz

```
interface Device {
    void print();
    void scan();
    void fax();
}
class SimplePrinter implements Device {
    public void print() { System.out.println("Printing..."); }
    public void scan() { throw new UnsupportedOperationException(); }
    public void fax()  { throw new UnsupportedOperationException(); }
}
```

1. ISP (Interface Segregation)
2. Problem: Device forces SimplePrinter to implement scan()/fax().
3. Fix: Split interfaces.

```
interface Printer { void print(); }
interface Scanner { void scan(); }
interface Fax { void fax(); }

class SimplePrinter implements Printer { public void print(){ /*...*/ } }
class MultiFunction implements Printer, Scanner, Fax { /*...*/ }
```

Quiz

```
class EmailSender {
    void send(String msg) { System.out.println("Sending: " + msg); }
}
class HRNotifier {
    private EmailSender sender = new EmailSender();
    void notifyHR() { sender.send("Report sent!"); }
}
```

1. DIP (Dependency Inversion)
2. Problem: HRNotifier depends on concrete EmailSender.
3. Fix: Depend on an abstraction + inject it.

```
interface MessageSender { void send(String msg); }
class SmtpSender implements MessageSender { public void send(String m){ /*...*/ } }
class ImapSender implements MessageSender { public void send(String m){ /*...*/ } }


class HRNotifier {
  private final MessageSender sender;
  HRNotifier(MessageSender s){ this.sender = s; }  // constructor injection
  void notifyHR(){ sender.send("Report sent!"); }
}
```

Chapter 5: Introduction to Design Patterns (Part I)

- History
- Object-oriented design principles
- Design patterns

The concept of design patterns

Design patterns

It is a summary of the experience of code design that has been used repeatedly and cataloged by classification. It describes some of the recurring problems in the software design process, and the core solutions to those problems. The purpose is to improve the reusability of the code, the readability of the code, and the reliability of the code.

The essence of design patterns is the practical application of object-oriented design principles, and a full understanding of the encapsulation, inheritance, and polymorphism of classes, as well as the relational and combinatorial relationships of classes.

The concept of design patterns

The four essential elements of a design pattern

element	description
Pattern Name	A mnemonic phrase that uses one or two words to describe the problem, solution, and effect of the pattern.
Problem	Describes the application environment of the pattern, i.e., when to use it. It explains the design problem and the causes and consequences of its existence, as well as a set of prerequisites that must be met.
Solution	The components of the design, their interrelationships, and their respective responsibilities and ways of working together are described. The solution does not describe a specific and specific design or implementation, but rather provides an abstract description of the design problem and how to solve it with a general combination of elements (a combination of classes or objects).
Consequence	It describes the effects of the application of the pattern and the trade-offs that should be made when using the pattern, i.e., the advantages and disadvantages of the pattern. It is mainly the measurement of time and space, as well as the impact of the model on the flexibility, scalability, and portability of the system.

The benefits of design patterns

- Reuse design: Reusing design makes more sense than reusing code, and it automatically leads to code reuse.
- To provide a common vocabulary for design, each pattern name is a design vocabulary, and the concept is that it is easier for programmers to communicate with each other.
- Adopting a schema vocabulary in your development documentation makes it easier for others to understand your ideas and practices, and makes it easier to write development documentation.
- Applying design patterns makes it easy to refactor the system, ensures that the right code is developed, and reduces the likelihood of errors in the design or implementation.
- Support for variation, which can provide a good system architecture for rewriting other applications.
- When used correctly, you can save a lot of time.

The categorization of design patterns

- Categorized by purpose (what the pattern is used to accomplish)
  - Creational patterns : Used to describe "how to create objects", its main feature is "separating the creation and use of objects".
  - Structural patterns : Used to describe how a class or object can be arranged into larger structures.
  - Behavioral patterns: Used to describe how classes or objects work together to accomplish tasks that no single object can do alone, and how responsibilities are assigned.
- Categorized by scope (patterns are mainly used for classes or objects)
  - Class patterns : Used to handle relationships between classes and subclasses, which are established through inheritance, are static, and are determined at compile time.
  - Object patterns : Used to deal with relationships between objects, which can be achieved by combining or aggregating, and can be changed at the runtime, making them more dynamic.

Classification of design patterns

		Purpose		
		Creational	Structural	Behavioral
Scope	class	**Factory Method**	**Adapters (Classes)**	Template method, Interpreter
	object	Abstract factory Builder Prototype **Singleton**	Adapter (Object) Proxy Composite **Decorator** Flyweight Facade Bridge	Chain of Responsibility Iterator Strategy Mediator **Observer** **Command** State Memento Visitor

Chapter 5
Introduction to Design Patterns Part II

Chapter 5: Introduction to Design Patterns
Part II

- Typical design patterns
- Singleton pattern
- Simple factory
- Factory method
- Abstract factory

Singleton

- The Singleton pattern ensures a class has only one instance and provides a global way to access it.
- Advantages
  - Reduced memory usage
  - Avoidance of conflicting state

Factory Method

- The Factory Method pattern lets a class defer object creation to its subclasses, allowing them to decide which specific type of object to instantiate.
- Advantages
  - Promotes loose coupling between classes
  - Makes code more extensible for adding new product types
  - Centralizes object creation logic

Adapters

- The Adapter pattern allows incompatible interfaces to work together by acting as a bridge that converts one interface into another that clients expect.
- Advantages
  - Enables reusability of existing classes
  - Allows integration of legacy systems without modifying their source code

Decorator

- The Decorator pattern dynamically adds new responsibilities to an object without altering its structure, by wrapping it with additional behavior.
- Advantages
  - Provides flexible alternative to subclassing for extending functionality
  - Follows open-closed principle
  - Allows mix-and-match of features at runtime

Observer

- The Observer pattern defines a one-to-many relationship where when one object changes state, all its dependents are automatically notified and updated.
- Advantages
  - Promotes loose coupling between subjects and observers
  - Enables dynamic relationships at runtime
  - Ensures consistency between related objects without tight dependencies

Command

- Command Pattern turns requests into objects that can be stored, queued, and undone.
- Advantages
  - Undo/Redo - Commands can remember state for reversal
  - Queueing - Commands can be stored and executed later
  - Decoupling - The invoker doesn't need to know request details
  - Extensible - Easy to add new commands without changing existing code

Quiz – Which design pattern should be used?

- Problem: An application needs a configuration object that must be accessible from anywhere in the code, but we want to ensure thread safety.
- Singleton Pattern

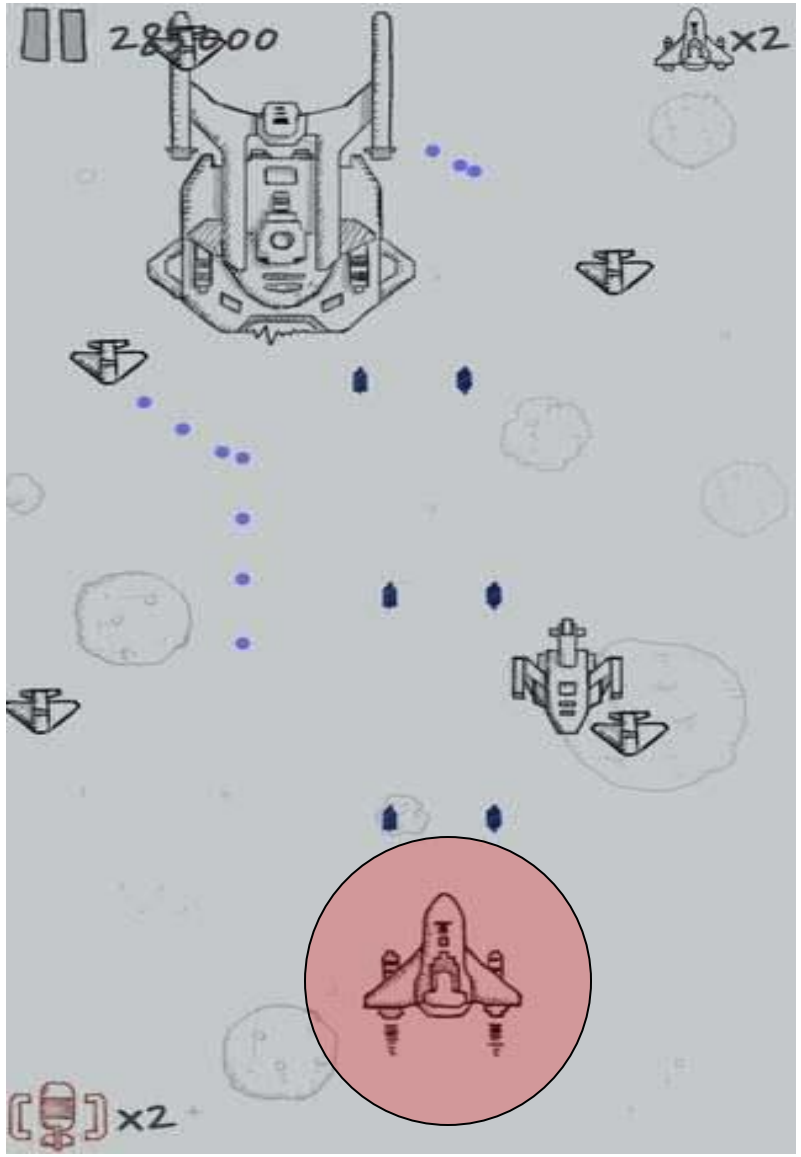
Quiz – Which design pattern should be used?

- Problem: A rich text editor where users can apply multiple formatting options (bold, italic, underline, color) in any combination.
- Decorator Pattern

Quiz – Which design pattern should be used?

- Problem: You're building an order system where when an order is placed, multiple actions must happen:
  - Update inventory
  - Send confirmation email
  - Process payment
  - Notify shipping department
  - Update customer loyalty points

Observer Pattern



Singleton pattern

Singleton pattern

- **Singleton pattern**

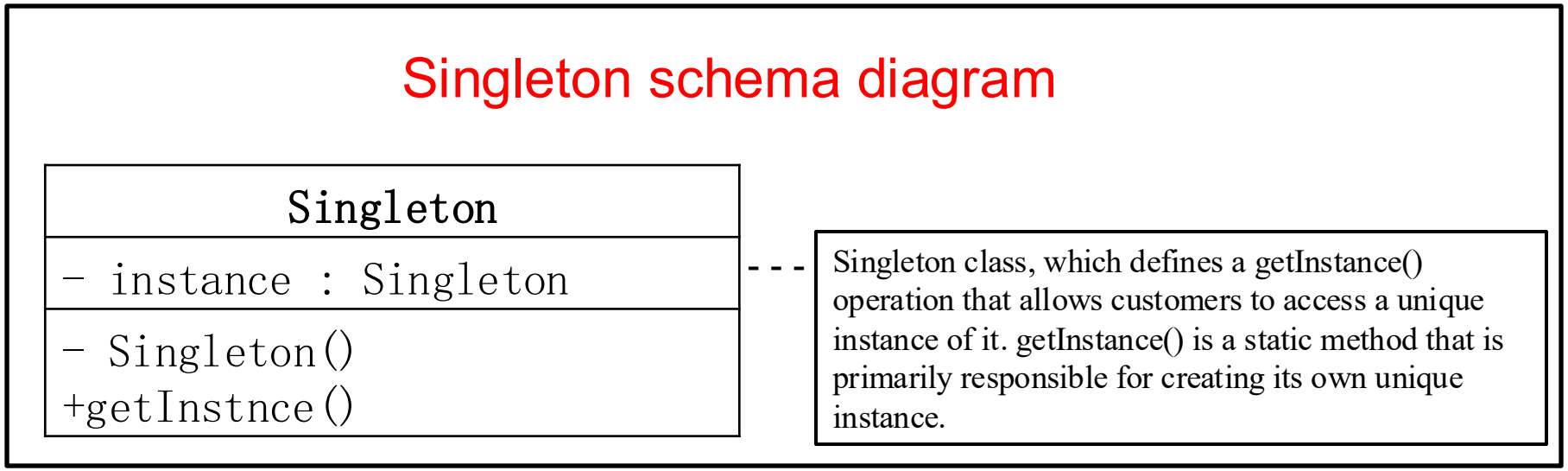
Ensure that there is only one instance of a class and provide a global access point to access it.

- Many times we have to make sure that there is only one instance of a class
  - There is only one instance of connection in the entire application that connects to the database
  - The operating system has only one clock
  - There is only one hero in the aircraft war game…

Singleton pattern

How do you ensure that there is only one instance of a class, and that this instance is easily accessible?

A global variable makes an object globally accessible, but it doesn't prevent you from instantiating multiple objects. One of the best ways to do this is to have the class itself responsible for saving its unique instance, which guarantees that no other instances can be created, and that it can provide a way to access that instance.



Singleton pattern (1): Eager initialization

Implemented in the classic singleton pattern

The eager initialization is instantiated immediately when the class is loaded, and only one instance will appear when it is used in the future.

Singleton class, which defines a getInstance() operation that allows customers to access a unique instance of it.

```
public class Singleton{
    private static Singleton singleton = new Singleton ();

    private Singleton(){
    }

    public static Singleton GetInstance () {
      return singleton;
    }
}
```

1. When the class is initialized, the object is loaded immediately, which is a natural linear security

2. Declare the constructor as private so that the outside world cannot use new to create such instances

3. Methods do not need to be synchronized, and the call efficiency is high

Singleton pattern (2): Lazy initialization

Implemented in the classic singleton pattern

Instead of instantiating the class when it is loaded, it is instantiated when the specified instance method is called, so that it is not instantiated when you don't want to use it. Generally more efficient than the Eager initialization.

Singleton class, which defines a getInstance() operation that allows customers to access a unique instance of it. getInstance() is a static method that is primarily responsible for creating its own unique instance.

```
public class Singleton {
    private static Singleton instance;
    private Singleton (){}

    public static Singleton getInstance() {
    if (instance == null) {
        instance = new Singleton();
    }
    return instance;
    }
}
```

1. Use a private static variable to record a unique instance of the Singleton class

2. Declare the constructor as private so that the outside world cannot use new to create such instances

3. Use the public static method getInstance() to instantiate the object and return that unique instance

Singleton Pattern (3): Thread-Safe Lazy Initialization

If multiple threads access the Singleton class at the same time, call the getInstance() method, it is possible to create multiple instances, how to avoid it?

By adding the synchronized keyword to the getInstance() method, each thread is forced to wait for other threads to leave the method before entering the method. That is, no two threads can enter the method at the same time.

```
public class Singleton {
    private static Singleton instance;
    private Singleton (){}
    public static synchronized Singleton getInstance() {
    if (instance == null) {
        instance = new Singleton();
    }
    return instance;
    }
}
```

Thread-safe, but inefficient!

Singleton pattern

element	description
Pattern Name	**Singleton Pattern**
Intent	You just want to have an object that can be accessed from all parts of the system, without using global variables and without passing references to objects.
Problem	How can I ensure that several different customer objects want to reference the same object?
Solution	A unique instance is guaranteed 1) Define a private static member variable to save a reference to a single instance; 2) define a public static method getInstance() to get a unique instance; 3) The class is responsible for instantiating the object "on first use".
Consequence	Advantages: 1) Controlled access to unique instances; 2) There is only one instance in memory, which reduces the memory overhead, avoids frequent creation and destruction of objects, and improves performance. 3) avoid multiple occupation of shared resources; 4) allow a variable number of instances; 5) Can be accessed globally.

Chapter 5: Introduction to Design Patterns
Part II

- Typical design patterns
- Singleton pattern
- Simple factory
- Factory method
- Abstract factory

Factory method

Background of the case

- (1) In the era when there was no factory, if the customer needed a BMW, then the customer needed to create a BMW car and then use it.
- (2) Simple factory: Later, the factory appeared, and the user no longer needed to create a BMW car, it was created by the factory, and what car he wanted could be created directly through the factory. For example, if you want a 320i series, the factory creates this series of cars.
- (3) Factory method: In order to meet customers, there are more and more BMW car series, such as 320i, 523i and so on, a factory can not create all BMW series, so it is divided into a number of specific factories, and each specific factory creates a series, that is, the specific factory class can only create a specific product. But the BMW factory is still an abstraction, and you need to specify a specific factory to produce the car.

Background of the case (no design pattern)

```
// Customer class directly creates BMW objects
public class Customer {
    public void buyCar() {
        // Customer has to know how to create specific BMW models
        BMW320i car1 = new BMW320i();
        BMW523i car2 = new BMW523i();

        // Customer is tightly coupled to specific BMW classes
        car1.start();
        car2.start();
    }
}

// Specific BMW classes
public class BMW320i {
    public void start() {
        System.out.println("BMW 320i started");
    }
}

public class BMW523i {
    public void start() {
        System.out.println("BMW 523i started");
    }
}
```

Simple factory

- The user needs to know how to create a car, so that the customer and the car are tightly coupled together.
- In order to reduce the coupling, there is a simple factory mode, which puts the details of the operation of creating the BMW into the factory, and the customer directly uses the factory creation method, and passes in the desired BMW car model, without having to know the details of the creation.

Simple factory

```java
// Simple Factory class
public class BMWFactory {
    public static BMW createCar(String model) {
        if (model.equals("320i")) {
            return new BMW320i();
        } else if (model.equals("523i")) {
            return new BMW523i();
        }
        throw new IllegalArgumentException("Unknown BMW model: " + model);
    }
}

// Customer class now uses factory
public class Customer {
    public void buyCar() {
        // Customer only needs to know the factory and model name
        BMW car1 = BMWFactory.createCar("320i");
        BMW car2 = BMWFactory.createCar("523i");

        // Customer works with BMW interface, not specific classes
        car1.start();
        car2.start();
    }
}
```

```java
// BMW interface
public interface BMW {
    void start();
}

// Specific BMW classes implement the interface
public class BMW320i implements BMW {
    public void start() {
        System.out.println("BMW 320i started");
    }
}

public class BMW523i implements BMW {
    public void start() {
        System.out.println("BMW 523i started");
    }
}
```

Simple factory

The simple factory mode provides a special factory class for creating objects, realizes the separation of responsibilities between object creation and use, the client does not need to know the class name and creation process of the specific product class created, only needs to know the parameters corresponding to the specific product class, and by introducing the configuration file, the new specific product class can be replaced and added without modifying any client code, which improves the flexibility of the system to a certain extent.

However, the disadvantage is that it does not comply with the "open-closed principle", and every time a new product is added, the factory class needs to be modified. When there are many product types, it may cause the factory logic to be too complex, which is not conducive to the expansion and maintenance of the system, and the factory class concentrates all the product creation logic, once it does not work normally, the entire system will be affected.

To solve the problem of the simple factory mode, the factory method pattern emerged

Factory Method

The Factory Method pattern abstracts the factory and defines an interface for creating objects.

Each time a new product is added, only the product and the corresponding concrete implementation factory class need to be added, and the specific factory class determines which product to instantiate, and the creation and instantiation of the object is deferred to the subclass, so that the design of the factory conforms to the "open and closed principle", and the original code does not need to be modified when expanding.

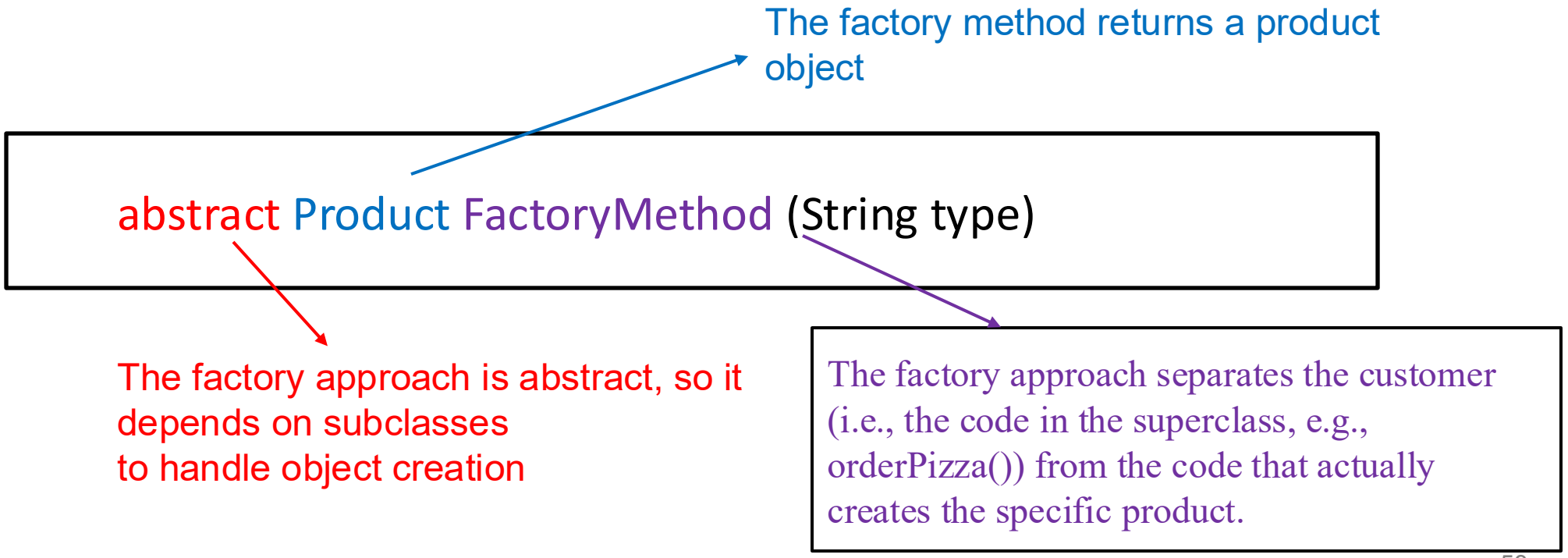
However, the disadvantage is that each additional product needs to add a specific product class and the implementation factory class, which multiplies the number of classes in the system, increases the complexity of the system to a certain extent, and also increases the dependence of the specific classes of the system

The Simple Factory isn’t really a design pattern, but the Factory Method is built on top of it.

Factory Method

Factory Method

It is used to handle the creation of objects and encapsulate such behavior in subclasses. In this way, the code in the client program about superclasses is decoupled from the code for creating subclass objects.



Factory method

```java
// Abstract Factory interface
public interface BMWFactory {
    BMW createCar();
}

// Concrete Factory for BMW 320i
public class BMW320iFactory implements BMWFactory {
    public BMW createCar() {
        return new BMW320i();
    }
}

// Concrete Factory for BMW 523i
public class BMW523iFactory implements BMWFactory {
    public BMW createCar() {
        return new BMW523i();
    }
}

// BMW interface (same as before)
public interface BMW {
    void start();
}
```

```java
// Concrete BMW classes (same as before)
public class BMW320i implements BMW {
    public void start() {
        System.out.println("BMW 320i started");
    }
}

public class BMW523i implements BMW {
    public void start() {
        System.out.println("BMW 523i started");
    }
}

// Customer class uses specific factories
public class Customer {
    public void buyCar() {
        // Each factory creates its specific product
        BMWFactory factory320i = new BMW320iFactory();
        BMWFactory factory523i = new BMW523iFactory();

        BMW car1 = factory320i.createCar();
        BMW car2 = factory523i.createCar();

        car1.start();
        car2.start();
    }
}
```

Simple Factory Mode VS Factory Method

- Simple factory
  - In Simple Factory, all the creation logic is centralized in one factory class. The client doesn't need to know about specific product classes - it just passes a parameter to the factory and gets back the appropriate product.
  - However, this approach violates the Open-Closed Principle because every time you add a new product type, you must modify the factory class. This becomes problematic as the system grows and you have many product types.
- Factory method
  - Factory Method follows the Open-Closed Principle by allowing you to add new products through extension rather than modification. You create new product classes and their corresponding factory classes without touching existing code.
  - The trade-off is that the client code needs to know about specific factory classes. The decision logic moves from the factory to the client - instead of the factory deciding what to create based on a parameter, the client decides which factory to use.

Abstract factory

Background of the case

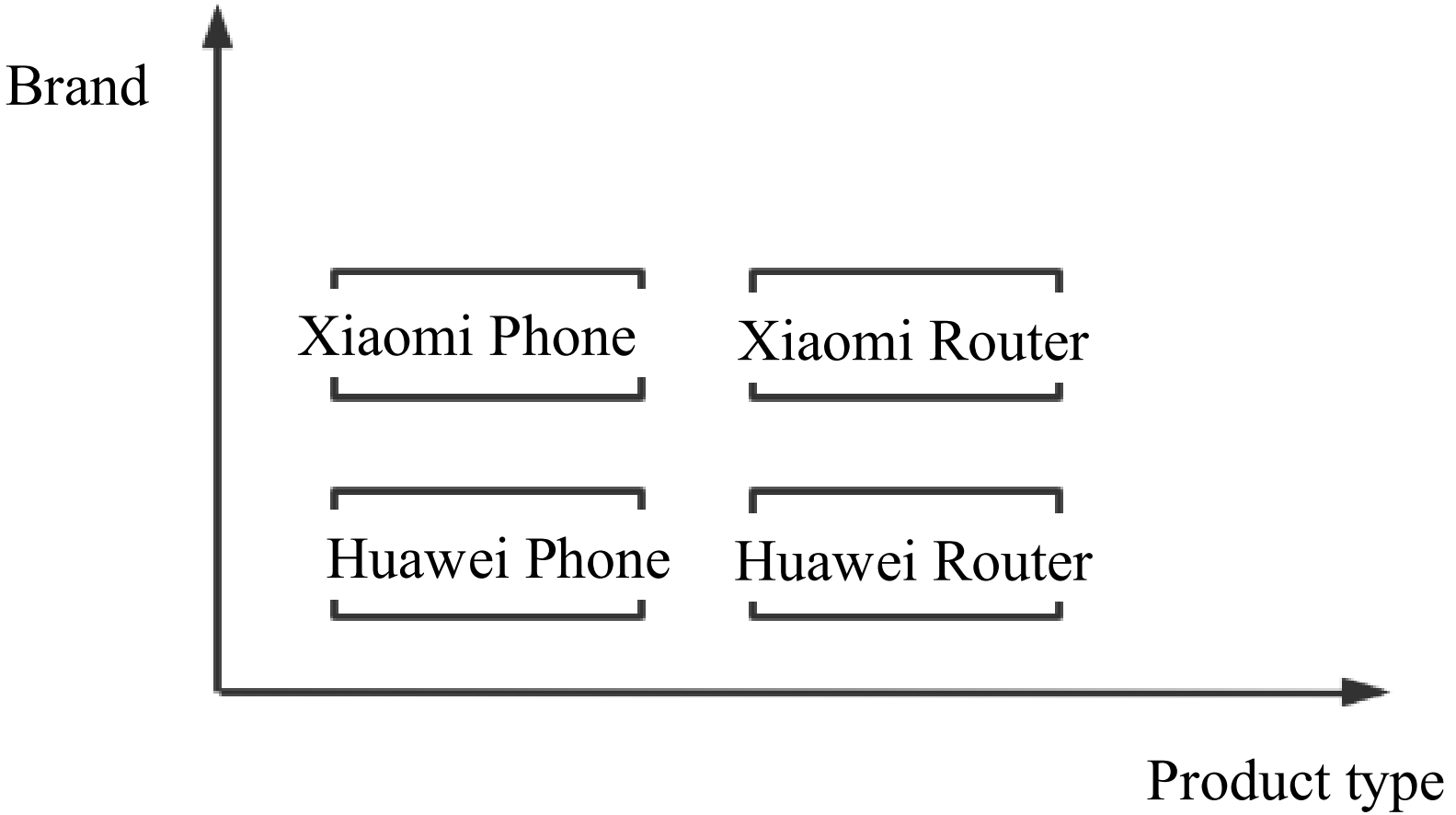
- **Understand the brand and the product**

  - Brand (product family): All products under a brand, such as mobile phones, routers, and computers under Huawei, are called Huawei's product families.

  - Product type: The same product under multiple brands, such as Huawei and Xiaomi's mobile phones under is called one product.

Background of the case

- **Understand the brand and the product**



Background of the case

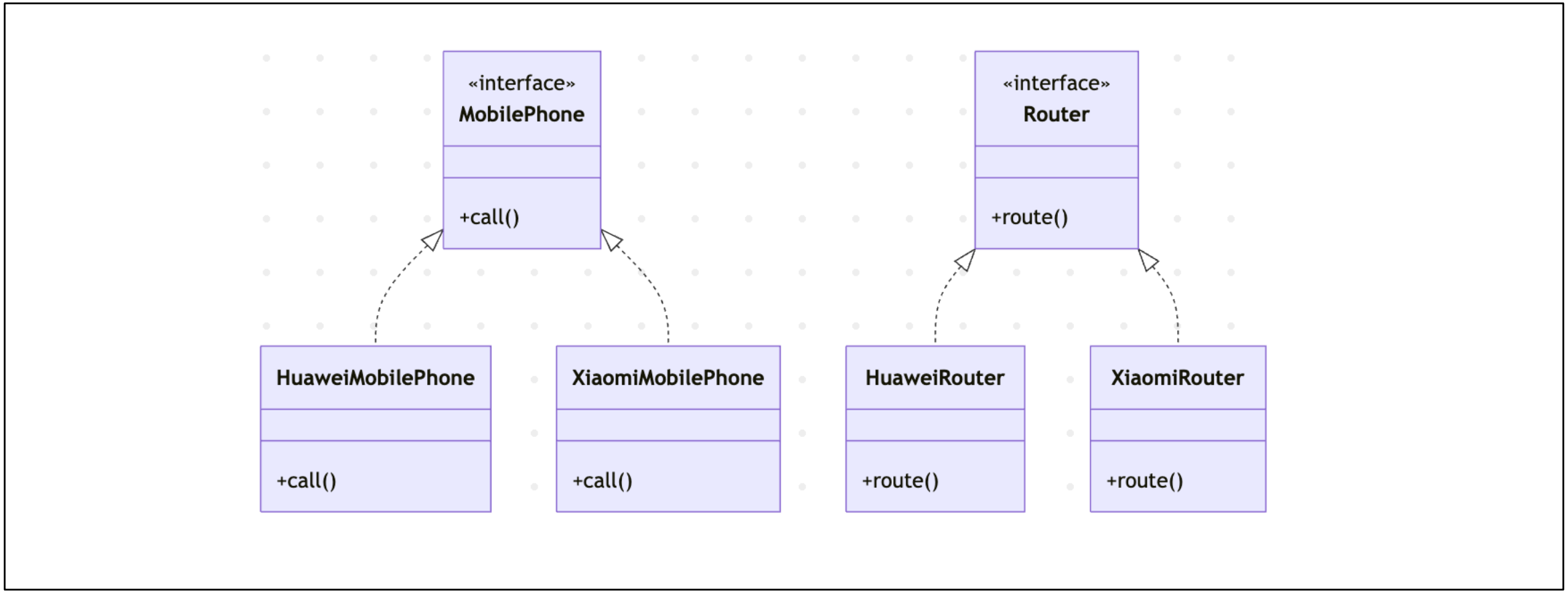
- There are two kinds of products, mobile phones and routers, and there are two brands, Huawei and Xiaomi, both of which can produce mobile phones and routers；
  - There are two products, mobile phone and router, and 2 interfaces are defined;
  - Both Xiaomi and Huawei can produce both, so there are 4 implementation classes；
- Now you need to create the factory class of Huawei and Xiaomi, first abstract the factory class, there is a method to create two products, and the interface class of the product is returned;
- Create factory implementation classes for Huawei and Xiaomi, inherit factory class interfaces, and implement methods for creating their respective products;
- When the client calls, directly use the factory interface class to create the required factory and get the corresponding product;

An overview of the Abstract Factory pattern

- The Abstract Factory Pattern is the creation of other factories around one gigafactory. This gigafactory is also known as a factory for other factories. This type of design pattern is a creative pattern that provides a way to create objects.

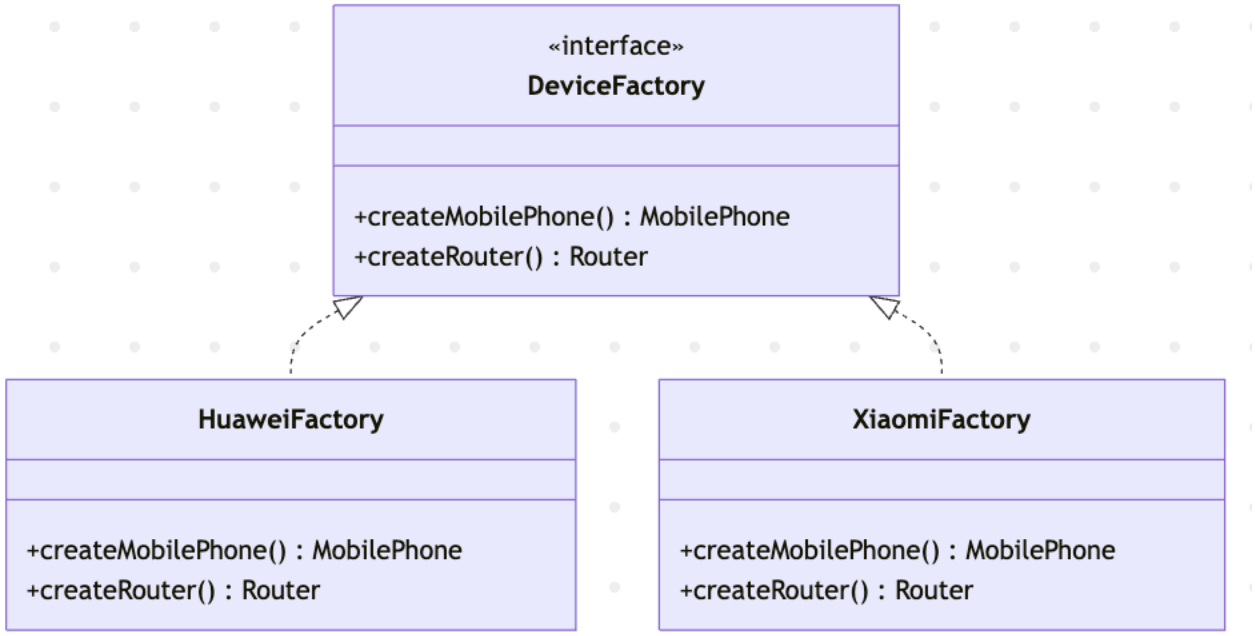
Abstract factory pattern

Class Diagram Analysis:



Abstract factory pattern

Class Diagram Analysis:



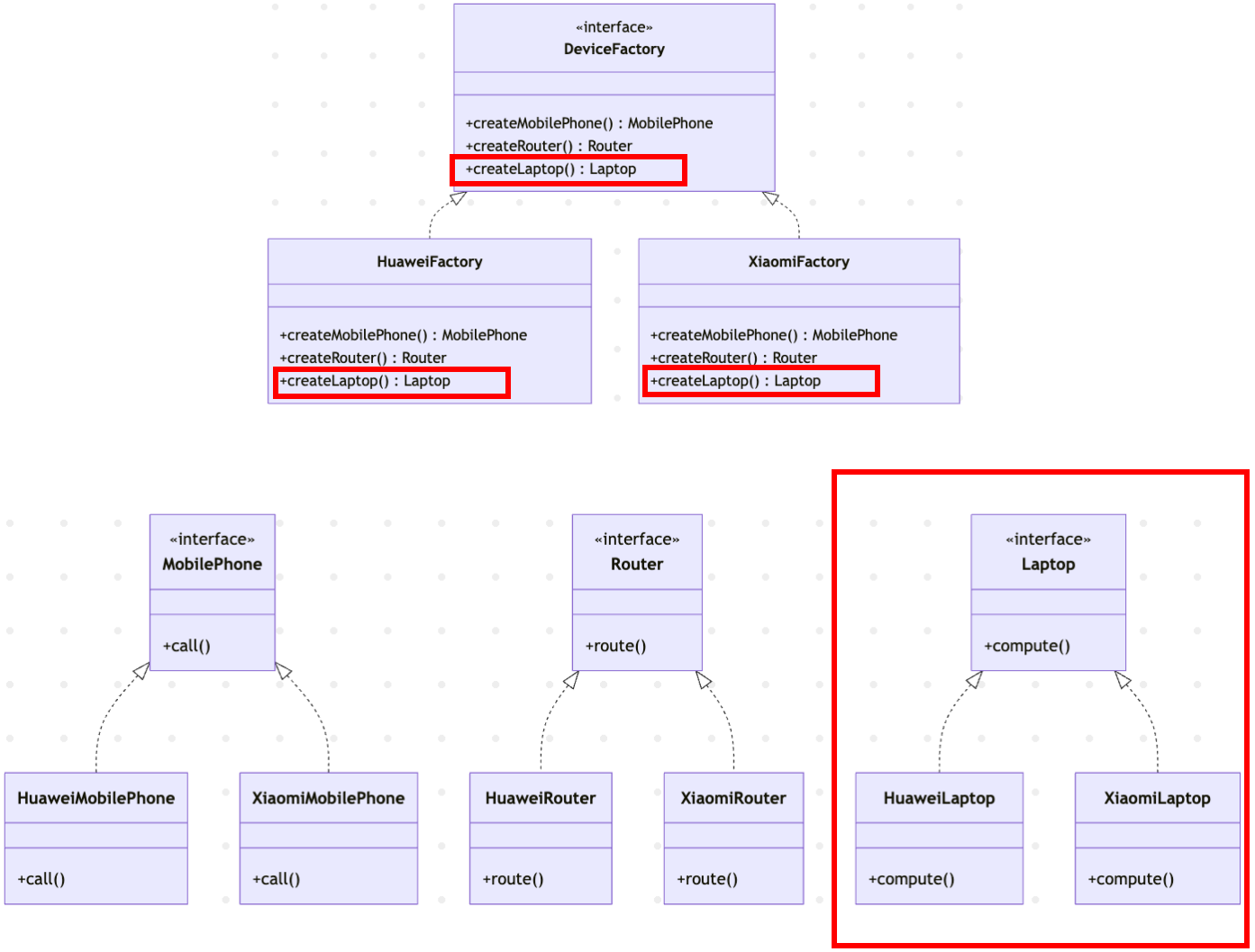
Abstract factory pattern (expand a product)

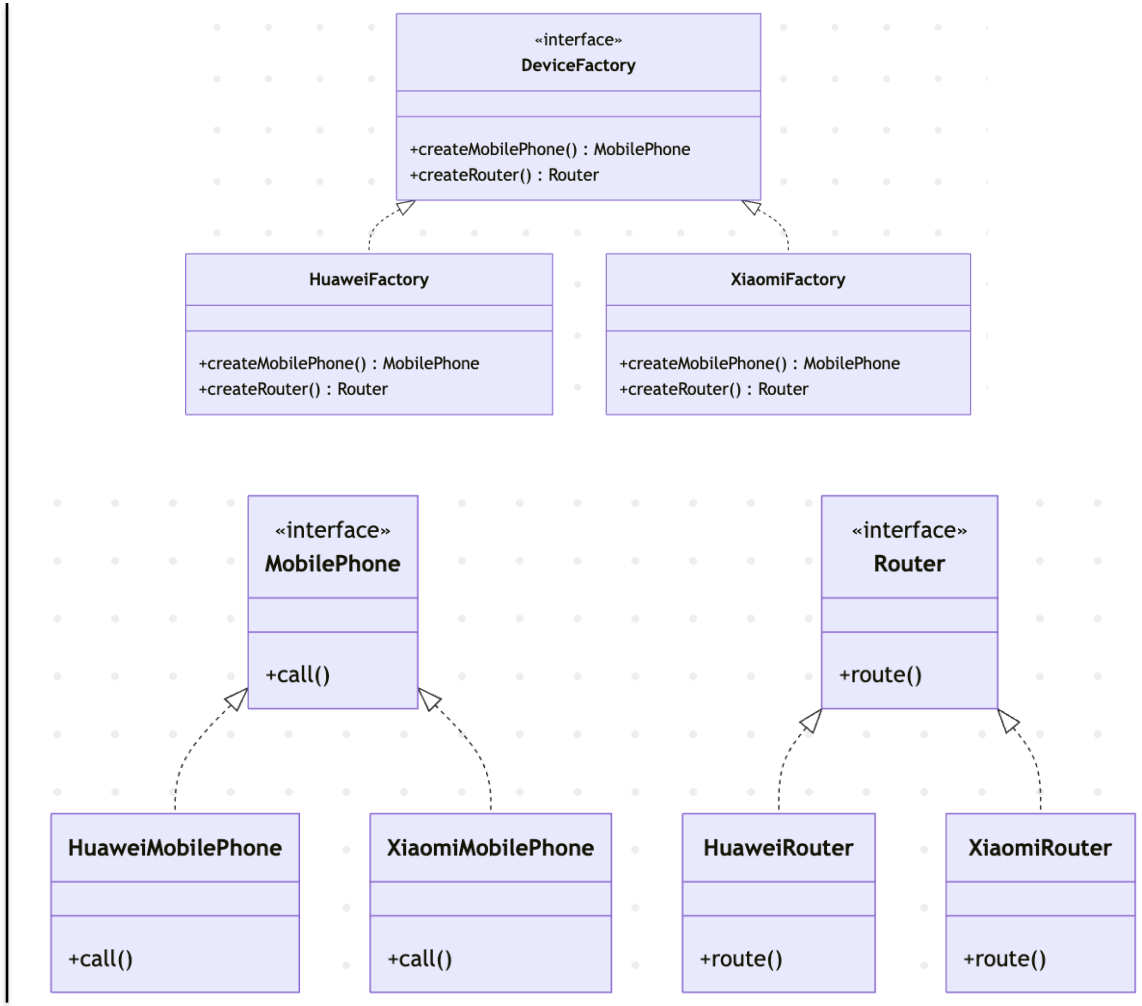
Now looking back at the concept of product family and product level, what happens if you add a new brand product family or product type?

Expand a product type

- We will find that it is very difficult to expand a product type, for example, adding laptops to the product type, which means that Huawei and Xiaomi can now produce computers, as shown in the picture on the following page (the red box is what needs to be done to add a new product type), and the factory interface class at the top level also needs to be modified, which is very troublesome.

Abstract factory pattern (expand a product)



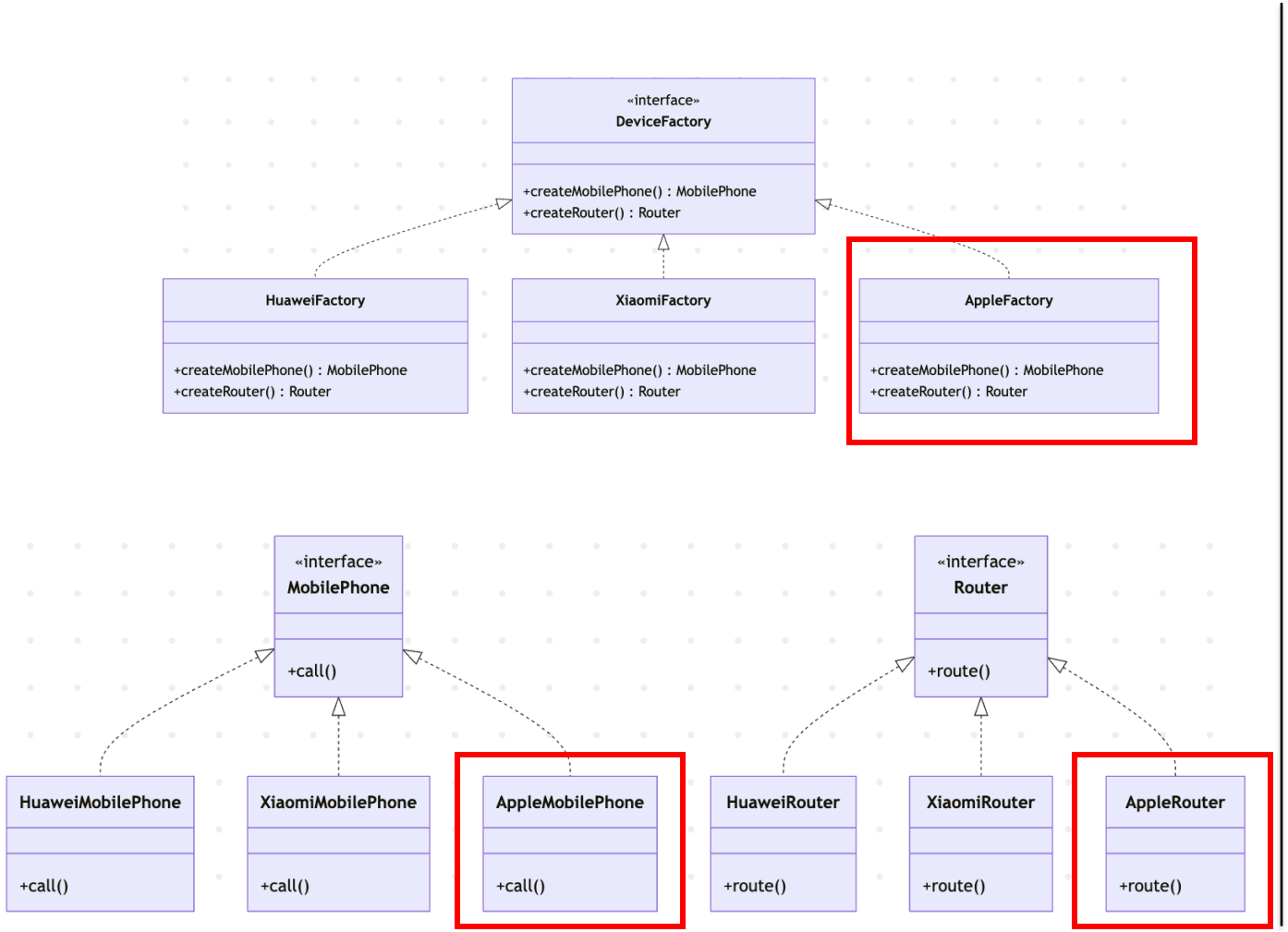


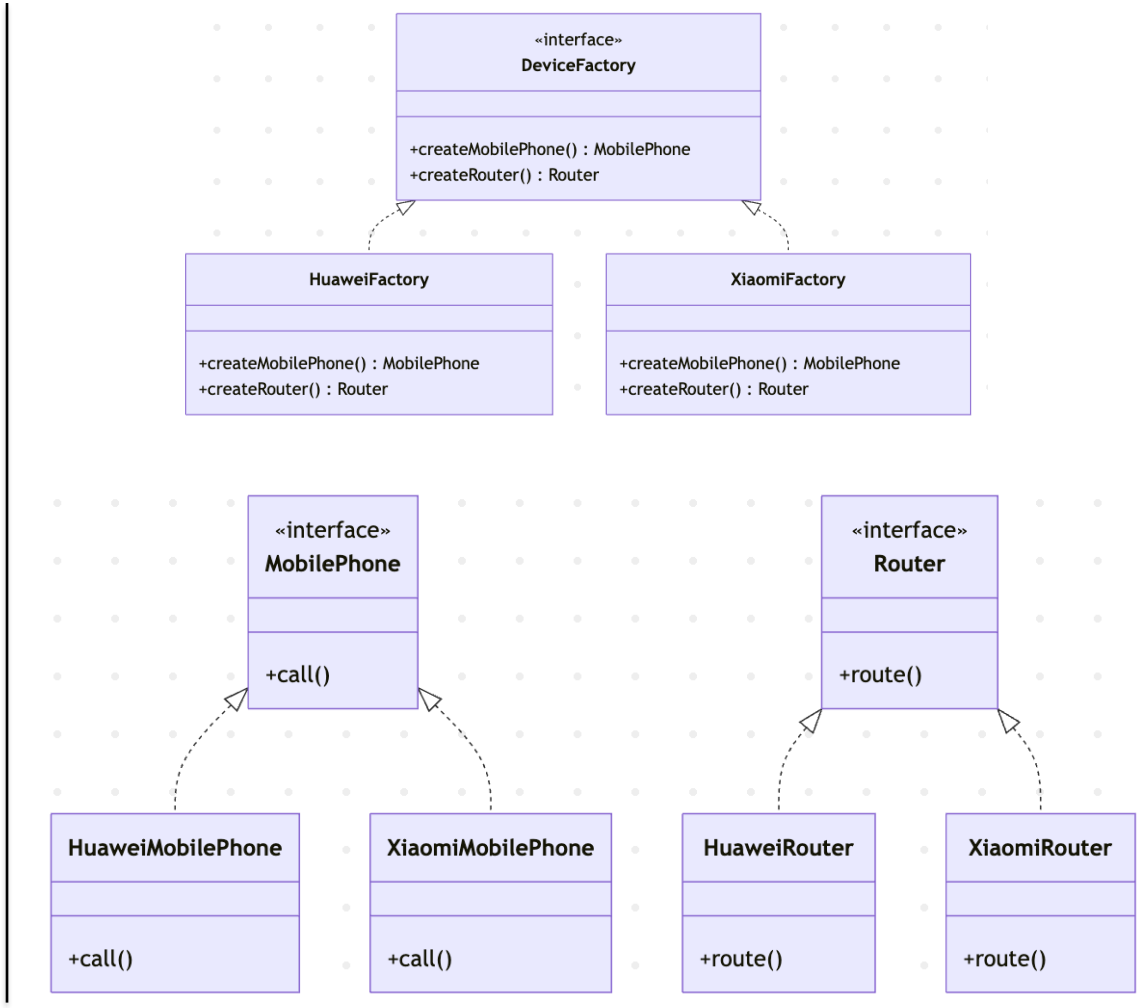
Abstract factory pattern (expand a brand)

Expand a brand (product family)

- If you add a new brand, that is, add a new manufacturer to produce mobile phones and routers, as shown in the picture on the following page (red box is what you need to do to add a new product family), adding a new product family does not need to modify the original code, which is in line with the OCP principle, which is very convenient.

Abstract factory pattern (expand a brand)





Advantages and disadvantages of the abstract factory pattern

- Advantages
  - When multiple objects in a product family are designed to work together, it ensures that the client always uses only objects from the same product family (creating products from one brand together).
- Disadvantages
  - It is very difficult to extend the product type, and to add a new product class, you need to modify both the code in the factory abstract class and the code in the specific implementation class.
  - It increases the abstraction and difficulty of understanding the system.

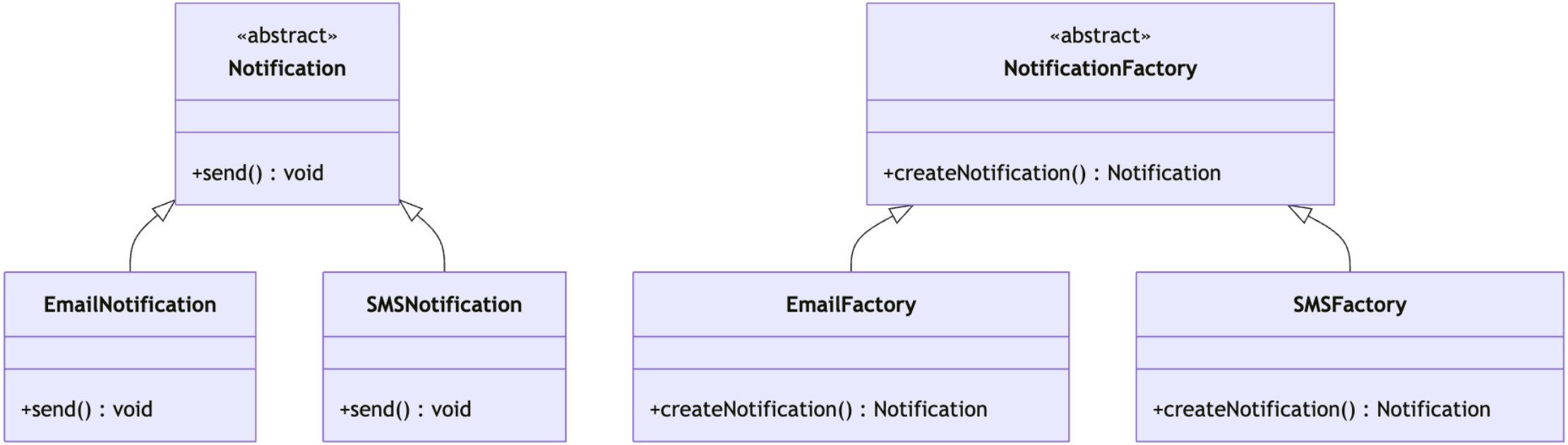
Quiz

Question:
You are designing a simple **Notification system** that can send messages by **Email** or **SMS**. Use one of the factory method to design the system.

Hints:

- No if-else or switch in the client code.
- You should be able to add a new notification type later without changing old code.

Quiz


«abstract»
Notification
+send() : void
EmailNotification
+send() : void
SMSNotification
+send() : void
«abstract»
NotificationFactory
+createNotification() : Notification
EmailFactory
+createNotification() : Notification
SMSFactory
+createNotification() : Notification

```
abstract class Notification {
    abstract void send();
}
class EmailNotification extends Notification {
    void send() { System.out.println("Sending Email Notification"); }
}
class SMSNotification extends Notification {
    void send() { System.out.println("Sending SMS Notification"); }
}
abstract class NotificationFactory {
    abstract Notification createNotification();
}
class EmailFactory extends NotificationFactory {
    Notification createNotification() { return new EmailNotification(); }
}
class SMSFactory extends NotificationFactory {
    Notification createNotification() { return new SMSNotification(); }
}
public class Main {
    public static void main(String[] args) {
        NotificationFactory factory = new EmailFactory();
        Notification n = factory.createNotification();
        n.send();
    }
}
```

Chapter 5: Introduction to Design Patterns
Part II

- Typical design patterns
- Singleton pattern
- Simple factory mode
- Factory method
- Abstract factory design pattern